ECG Classification — Viva Preparation Document

This document prepares you to explain your ECG classification project end-to-end for a viva. It is structured for quick revision and contains explanations, formulas, practical tips, and ready answers to common examiner questions.

---

1. Project Overview

- **Problem statement:** Build a deep-learning system to classify ECG recordings (or ECG-derived images) into clinically relevant classes (e.g., normal, myocardial infarction, arrhythmia variants) using convolutional neural networks (EfficientNet and DenseNet).
- **Motivation:** Automated ECG interpretation can speed diagnosis, reduce clinician workload, enable continuous remote monitoring, and bring diagnostic capability to low-resource settings. Deep learning models can learn discriminative morphological and temporal patterns that are difficult to capture with hand-crafted rules.
- **Real-world applications:** Emergency triage, telemedicine, wearable analytics, hospital decision support, population screening, and clinical research.

2. Project Journey (Storytelling for Viva)

- **How the idea began:** The project idea grew from observing the burden on clinicians to read large numbers of ECGs and the potential for ML to assist diagnosis. Existing automated systems often struggle with heterogenous data; modern CNNs and transfer learning suggested a practical approach.
- **Challenges faced:** Data heterogeneity (different devices/sampling rates), strong class imbalance, noisy recordings and mislabeled data, overfitting on limited datasets, and choosing physiologically plausible augmentations.
- **Key decisions taken:** Convert signals to image representations for 2D CNN transfer learning (or use raw waveforms when appropriate), use pretrained EfficientNet and DenseNet backbones, apply stratified patient-wise splitting to avoid leakage, and adopt class-weighting/focal loss for imbalance.

3. Dataset Details

- **Type of dataset:** ECG signals (time-series) and/or ECG images (JPG/PNG). The project may use raw waveform formats (e.g., EDF, WFDB binary) or pre-rendered lead images.
- **Data source:** Public clinical datasets (e.g., PTB-XL, PhysioNet collections) and curated clinical images. In viva, cite the exact dataset files and versions you used.
- **Data distribution:** Typically skewed — many normal recordings, fewer pathological examples (MI, rarer arrhythmias). Provide exact class counts during viva.
- **Data issues:** missing/corrupt records, label noise (inter-annotator variability), class imbalance, baseline wander, muscle artifacts, and variable sampling rates and lead sets.

4. Data Preprocessing

- **Cleaning:** Remove or flag corrupt files, harmonize durations, and trim/pad to consistent length. Convert binary waveform formats to arrays and save canonical representations.
- **Handling missing values:** For short gaps, interpolate (linear or spline). For long missing segments, discard the sample or mark as unusable.
- **Outlier detection and removal:** Use per-record statistics (mean, std) and Z-score thresholds to detect out-of-range amplitude values. Verify outliers visually before removal; cap or remove only physiologically implausible extremes.
- **Normalization vs Standardization:**
- Normalization (min–max scaling): rescales values into [0,1].
- Formula: $$x' = \\\frac{x - \\\min(x)}{\\\max(x) - \\\min(x)}$$
- Standardization (z-score): centers data to zero mean and unit variance.
- Formula: $$x' = \\\frac{x - \\\mu}{\\\sigma}$$
- When to use: normalize for image inputs and pixel-range expectations; standardize for algorithms that assume zero mean/unit variance or when features have differing scales.
- **Data augmentation (if used):**
- Time-series augmentation: additive low-amplitude Gaussian noise, time-warping, scaling, random time-shift/cropping, frequency perturbation, lead dropout.
- Image augmentation: small rotations, brightness/contrast jitter, random crop/pad, limited elastic transforms. Always ensure physiological plausibility (avoid altering morphology that changes diagnosis).

5. Feature Engineering

- **Signal-derived features (optional):** Heart rate, RR-interval statistics (mean, sd), QRS duration, PR interval,

QT/QTc, ST deviation measures.
- **Signal processing steps:** Bandpass filtering (e.g., 0.5–40 Hz) to remove baseline wander and high-frequency noise; notch-filter at 50/60 Hz if power-line interference; denoising via wavelet thresholding when necessary.
- **Time–frequency transforms:** STFT or continuous wavelet transform (CWT) to generate scalograms used as CNN inputs for frequency-aware representations.

6. Model Architecture

- **EfficientNet (explain briefly):**
- Family of models that use MBConv blocks (inverted residuals + squeeze-and-excitation) and a compound scaling method that jointly scales depth, width, and resolution.
- Variants: B0..B7 trade off size vs accuracy. EfficientNet provides strong accuracy-per-parameter and is efficient for inference.
- **DenseNet (explain briefly):**
- Dense connectivity pattern where each layer receives as input the concatenation of all previous layers' feature maps.
- Growth rate determines how many new feature maps each layer adds; transition layers compress and pool.
- Encourages feature reuse, mitigates vanishing gradients.
- **Why these models:** Both benefit from transfer learning, strong visual feature extraction, and proven performance on medical-image-like tasks. EfficientNet for efficiency, DenseNet for feature reuse and gradient flow.
- **Differences:** EfficientNet uses compound scaling and MBConv blocks; DenseNet uses dense concatenation connections. EfficientNet is often more parameter-efficient; DenseNet increases feature-map dimensionality and memory usage but can converge stably.

7. Model Training

- **Training process:** Use transfer learning — initialize backbone with ImageNet weights, replace classification head with task-specific layers (global pooling → dropout → dense → softmax). Freeze early layers initially, fine-tune deeper layers later.
- **Train–validation–test split:** Typical splits are 70/15/15 or 80/10/10; use stratified splitting to preserve class proportions and perform patient-wise splitting to prevent data leakage.
- **Cross-validation:** Use stratified K-fold (e.g., 5-fold) when dataset is small; ensure patient-wise folds to avoid leakage.
- **Loss function:** Categorical cross-entropy for multi-class classification:
- $$L = -\\sum_{c} y_c \\log(\\hat y_c)$$
- For class imbalance, use class weights or focal loss to down-weight easy negatives.
- **Optimizer:** Adam (default) or SGD with momentum; use learning-rate schedulers (ReduceLROnPlateau, cosine annealing) and weight decay for regularization.

8. Hyperparameter Tuning

- **Learning rate:** Crucial hyperparameter. Typical ranges: Adam 1e-4–1e-3, SGD 1e-3–1e-1 (with momentum). Use an LR finder to pick a suitable starting point.
- **Batch size:** Chosen based on GPU memory. Typical: 8–64. Use gradient-accumulation if constrained.
- **Epochs:** Commonly 20–100; monitor validation metric and use early stopping.
- **Techniques:** Grid search for small discrete spaces; random search for broader spaces; Bayesian optimization (Optuna, Hyperopt) for efficient tuning. Combine with early stopping to reduce compute.

9. Evaluation Metrics

- **Accuracy:** $$\\text{Accuracy}=\\frac{TP+TN}{TP+TN+FP+FN}$$
- **Precision (per class):** $$\\text{Precision}=\\frac{TP}{TP+FP}$$
- **Recall (Sensitivity):** $$\\text{Recall}=\\frac{TP}{TP+FN}$$
- **F1-score:** $$F1=\\frac{2\\cdot \\\text{Precision}\\cdot \\\text{Recall}}{\\text{Precision}+\\text{Recall}}$$
- **Confusion matrix:** Use to inspect per-class confusions and systematic errors.
- **ROC-AUC:** Use for binary tasks or one-vs-rest multiclass evaluations. For rare positives, consider precision–recall AUC.
- **Metric choice guidance:** For imbalanced datasets, prefer macro-F1, class-wise recall/precision and detailed confusion analysis over raw accuracy.

10. Results and Analysis

- **Model comparison (EfficientNet vs DenseNet):**
- Compare per-class precision/recall, macro/micro F1, confusion matrices, ROC/PR curves, and inference metrics (latency, parameter count, memory).

- Typical tradeoffs: EfficientNet often offers a better accuracy-to-parameter ratio; DenseNet may provide stable training and feature reuse benefits. Exact results depend on dataset and tuning.
- **Final results:** In viva, present clear tables with test-set metrics and the confusion matrix, and report statistically significant gains if any.
- **Observations:** Analyze common confusions and root causes (e.g., label ambiguity, subtle morphology differences, noisy leads). Use Grad-CAM or saliency maps to verify that the model attends to clinically relevant regions.

11. Deployment / Implementation

- **How the model is used:** Typical pipeline — preprocess raw ECG → apply preprocessing/augmentation → feed to model → get predicted class probabilities → postprocess and map to clinical labels; provide visualization or explanation (Grad-CAM) for clinician review.
- **Tools/technologies:** Python, PyTorch/TensorFlow, Flask or FastAPI for REST inference service, Docker for containerization, ONNX/TensorRT for optimized inference; CI/CD for versioning.
- **Operational considerations:** Model versioning, input validation (sampling rate, channels), logging, latency targets, privacy and secure storage of patient data.

12. Limitations and Future Scope (detailed)

- **Limitations:**
- Dataset size and class imbalance limit generalization for rare conditions.
- Label noise and inter-observer variability reduce achievable ceiling performance.
- Domain shift between devices/sites affects robustness without domain adaptation.
- Image-only approaches may lose some temporal nuance present in waveforms.
- Explainability and clinician acceptance remain barriers for deployment.
- **Future scope:**
- Gather multi-center, multi-device datasets and apply domain adaptation.
- Use ensembles combining EfficientNet, DenseNet, and waveform models (1D-CNN, Transformer) to capture both spatial and temporal features.
- Explore multi-task learning (diagnosis, localization, severity) and semi-supervised methods for unlabeled data.
- Improve explainability via Grad-CAM, Integrated Gradients, and clinician-in-the-loop validation.
- Optimize for edge deployment (quantization/pruning) and run prospective clinical validation trials.

Viva-Focused Short Q&A (Ready Answers)

- Q: Why did you convert ECG signals to images rather than using raw waveforms?
- A: Image conversion allows reusing powerful 2D pretrained CNNs (transfer learning). Raw waveforms preserve temporal relationships; depending on dataset and compute, either approach can be valid. In this project the choice was guided by dataset format and the availability of pretrained 2D models.
- Q: How did you handle class imbalance?
- A: Stratified sampling, patient-wise splitting, class weights in the loss function, focal loss option, targeted augmentation for minority classes, and reporting macro-F1 and per-class recall.
- Q: How did you avoid patient leakage?
- A: All splits (train/val/test or CV folds) were patient-wise: records from a single patient stay in a single split.
- Q: Which metric is clinically most important?
- A: It depends: screening tools prioritize sensitivity (recall) to catch positives; diagnostic tools prioritize balanced precision and specificity. Macro-F1 is a robust overall metric for imbalanced multi-class tasks.
- Q: How can you trust the model's explanations?
- A: Use Grad-CAM/saliency maps and present examples to clinicians for validation; check that highlighted regions coincide with known morphological markers.

Practical Viva Tips

- Memorize exact dataset statistics and final test metrics (accuracy, macro-F1, per-class recall) — examiners often ask for numbers.
- Be ready to walk through one end-to-end example: raw recording → preprocessing → model input → prediction → Grad-CAM explanation.
- Explain why each preprocessing choice was made with physiological reasoning (e.g., bandpass 0.5–40 Hz removes baseline wander and muscle noise while preserving the diagnostic band).
- If asked about failures, show a few misclassified examples and explain plausible reasons (label noise, ambiguous morphology, noisy leads).

Next steps I can do for you

- Convert this to a PDF or slide deck for your viva.
- Populate the Results section with actual numbers and confusion matrices from your experiment outputs (I can read files in `outputs/` on request).

Tell me which option you prefer and I will proceed.